

PrediSched benchmark s20-20260927

Generated 2026-09-27 15:28 by scripts/make-report.py from results/benchmark/s20-20260927/. 125 runs: 5 scenarios × 5 strategies × 5 repetitions.

Headlines: predictive vs the best baseline

Per scenario, the best baseline is the reactive strategy with the best mean for that metric. A difference is only called one at $p < 0.05$ (two-sided Welch t-test over the repetitions; d is Cohen's d). ▲ predictive better, ▼ worse, ● no significant difference.

steady

- Mean latency: no significant difference vs Round robin (+7.4 %, $p = 0.712$)
- p95 latency: no significant difference vs Round robin (-1.4 %, $p = 0.918$)
- Throughput: no significant difference vs Round robin (-0.7 %, $p = 0.265$)
- SLA violations: 0 for every strategy (no difference)

bursty

- ▼ Mean latency: predictive +12.9 % vs Resource-aware ($p = 0.012$, $d = 2.69$): worse
- p95 latency: no significant difference vs Resource-aware (+7.3 %, $p = 0.070$)
- Throughput: no significant difference vs Least loaded (-0.0 %, $p = 0.551$)
- SLA violations: 0 for every strategy (no difference)

heterogeneous

- Mean latency: no significant difference vs Resource-aware (-2.2 %, $p = 0.694$)
- p95 latency: no significant difference vs Resource-aware (-6.3 %, $p = 0.462$)
- ▼ Throughput: predictive -0.3 % vs Resource-aware ($p = 0.020$, $d = -1.85$): worse
- SLA violations: 0 for every strategy (no difference)

mixed

- ▲ Mean latency: predictive -49.6 % vs Random ($p < 0.001$, $d = -5.46$): better
- ▲ p95 latency: predictive -25.1 % vs Random ($p = 0.002$, $d = -2.96$): better
- ▲ Throughput: predictive +5.0 % vs Random ($p = 0.007$, $d = 3.21$): better
- ▲ SLA violations: predictive -69.0 % vs Random ($p < 0.001$, $d = -4.66$): better

failure

- Mean latency: no significant difference vs Resource-aware (+29.5 %, $p = 0.270$)
- p95 latency: no significant difference vs Resource-aware (+38.0 %, $p = 0.347$)
- Throughput: no significant difference vs Random (-0.2 %, $p = 0.371$)
- SLA violations: no significant difference vs Round robin (+850.0 %, $p = 0.270$)

Setup and controls

- Every run starts a fresh cluster of real processes on one laptop (Intel i5-8350U, 4 cores / 8 threads, 15 W): one scheduler (configs/benchmark-node.yaml, PostgreSQL history on, result cache off, speculation off) and three workers. The prediction server and the mock HTTP target run for the whole suite.
- Each scenario replays one saved trace from workloads/benchmark, identical for every strategy and repetition (SHA-256 checked before every run). Trace seeds were used neither for the training dataset nor for λ tuning.
- Repetition-major order with the strategy order rotated each repetition, so machine drift spreads over strategies.
- The first 10 % of each trace's tasks are excluded from the latency percentiles (warm-up). Every task carries a 2000 ms deadline; SLA violation = failed or later than that.
- Latency is submit → completion as seen by the client (status polled every 50 ms).

Scenario	Profile	Pattern	Rate (/s)	Tasks	Seed	Workers	Fault
steady	mixed	steady	20	240	2001	homogeneous	

bursty	bursty	bursty	12	240	2002	homogeneous	
heterogeneous	mixed	steady	15	240	2003	heterogeneous	
mixed	long_tail	periodic	15	240	2004	heterogeneous	
failure	mixed	steady	15	240	2005	heterogeneous	kill worker-2 at 6 s

Results per scenario

Mean ± standard deviation over the repetitions. The last line gives predictive vs the best baseline for each column.

steady

Strategy	Mean latency	p95 latency	p99 latency	Throughput	Makespan	Imbalance (std of tasks/worker)
Round robin	343 ± 92	938 ± 211	1,172 ± 254	21.23 ± 0.03	11,306 ± 15	0.00 ± 0.00
Random	435 ± 150	1,054 ± 252	1,315 ± 233	21.22 ± 0.04	11,313 ± 21	6.16 ± 0.00
Least loaded	436 ± 102	1,134 ± 210	1,297 ± 200	21.20 ± 0.06	11,322 ± 33	16.58 ± 4.91
Resource-aware	515 ± 232	1,158 ± 291	1,323 ± 306	21.22 ± 0.03	11,313 ± 14	11.88 ± 3.50
Predictive	368 ± 116	925 ± 175	1,155 ± 199	21.07 ± 0.27	11,391 ± 147	10.11 ± 9.31
predictive vs best +7.4 % vs Round robin, p 0.712 -1.4 % vs Round robin, p 0.918 -1.4 % vs Round robin, p 0.911 -0.7 % vs Round robin, p 0.265 +0.7 % vs Round robin, p 0.267 +nan % vs Round robin, p 0.072						

bursty

Strategy	Mean latency	p95 latency	p99 latency	Throughput	Makespan	Imbalance (std of task
Round robin	111 ± 53	351 ± 135	449 ± 158	12.56 ± 0.03	19,104 ± 52	0.00 ± 0.00
Random	102 ± 30	334 ± 96	423 ± 75	12.57 ± 0.05	19,098 ± 79	6.16 ± 0.00
Least loaded	97 ± 18	318 ± 26	393 ± 25	12.59 ± 0.02	19,058 ± 24	27.12 ± 1.00
Resource-aware	87 ± 1	289 ± 19	381 ± 9	12.58 ± 0.01	19,072 ± 17	6.16 ± 0.00
Predictive	98 ± 6	310 ± 7	391 ± 16	12.59 ± 0.01	19,066 ± 13	82.12 ± 0.00
predictive vs best +12.9 % vs Resource-aware, p 0.012 +7.3 % vs Resource-aware, p 0.070 +2.7 % vs Resource-aware, p 0.263 -0.0 % vs Least loaded, p 0.551 +0.0 % vs Least loaded, p 0.553 +nan % vs Round robin, p 0.072						

heterogeneous

Strategy	Mean latency	p95 latency	p99 latency	Throughput	Makespan	Imbalance (std of t
Round robin	214 ± 8	635 ± 13	903 ± 14	14.50 ± 0.02	16,551 ± 21	0.00 ± 0.00
Random	250 ± 17	750 ± 72	981 ± 37	14.47 ± 0.03	16,585 ± 31	0.00 ± 0.00
Least loaded	190 ± 9	596 ± 37	983 ± 162	14.48 ± 0.02	16,571 ± 22	0.00 ± 0.00
Resource-aware	171 ± 9	547 ± 55	840 ± 39	14.51 ± 0.02	16,536 ± 20	0.00 ± 0.00
Predictive	167 ± 18	513 ± 82	766 ± 101	14.48 ± 0.02	16,578 ± 25	0.00 ± 0.00
predictive vs best -2.2 % vs Resource-aware, p 0.694 -6.3 % vs Resource-aware, p 0.462 -8.9 % vs Resource-aware, p 0.182 -0.3 % vs Resource-aware, p 0.020 +0.3 % vs Resource-aware, p 0.020 +nan % vs Round robin, p 0.072						

mixed

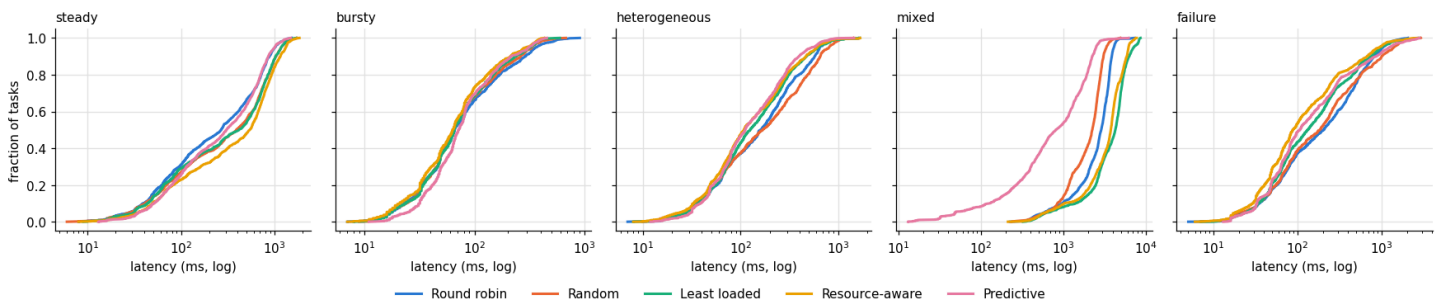
Strategy	Mean latency	p95 latency	p99 latency	Throughput	Makespan	Imbalance (std of tasks/worker)
Round robin	2,906 ± 193	4,057 ± 236	4,509 ± 171	12.48 ± 0.20	19,231 ± 312	3.96 ± 0.76
Random	2,225 ± 234	3,246 ± 312	3,978 ± 304	12.88 ± 0.28	18,634 ± 415	8.92 ± 0.80
Least loaded	4,383 ± 774	6,621 ± 1,223	7,045 ± 1,171	10.59 ± 0.29	22,686 ± 624	24.05 ± 13.01
Resource-aware	3,942 ± 419	6,222 ± 706	6,635 ± 612	11.06 ± 0.54	21,745 ± 1,042	18.59 ± 5.31
Predictive	1,121 ± 164	2,429 ± 233	2,970 ± 349	13.53 ± 0.02	17,741 ± 20	52.14 ± 8.35
predictive vs best -49.6 % vs Random, p < 0.001 -25.1 % vs Random, p 0.002 -25.3 % vs Random, p 0.001 +5.0 % vs Random, p 0.007 -4.8 % vs Random, p 0.008 +1217.1 % vs Round robin, p < 0.001 -18.2 % vs Random, p 0.002						

failure

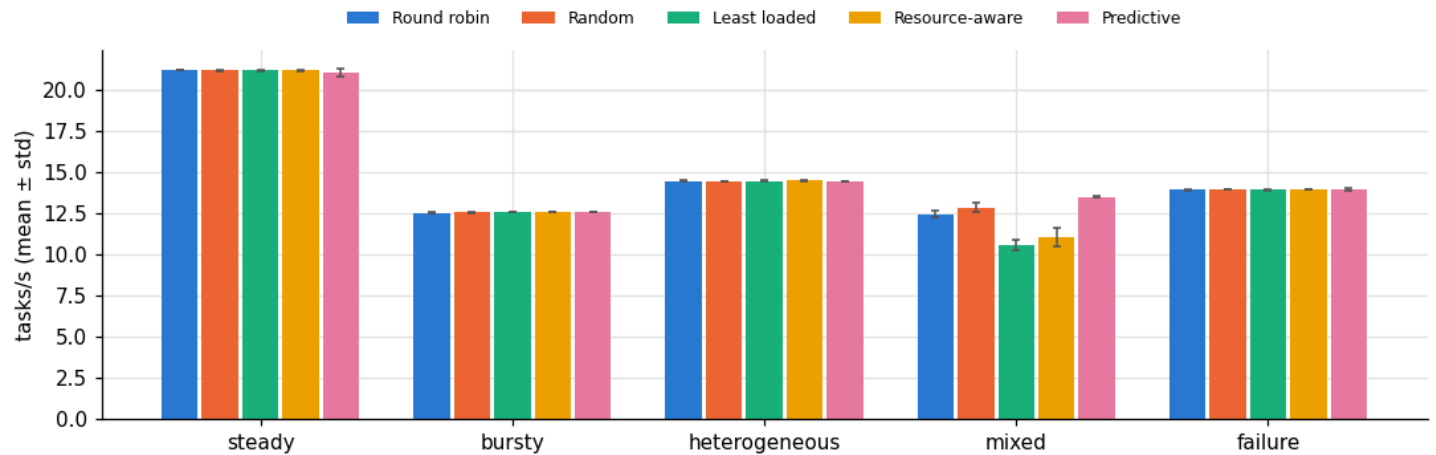
Strategy	Mean latency	p95 latency	p99 latency	Throughput	Makespan	Imbalance (std of tasks/wo
Round robin	271 ± 22	799 ± 92	1,301 ± 18	13.97 ± 0.03	17,184 ± 41	37.57 ± 0.00
Random	267 ± 36	957 ± 141	1,283 ± 179	14.00 ± 0.01	17,148 ± 18	34.92 ± 0.00
Least loaded	211 ± 50	738 ± 303	1,161 ± 312	13.97 ± 0.03	17,182 ± 40	35.90 ± 0.00
Resource-aware	160 ± 47	608 ± 271	1,038 ± 325	13.99 ± 0.02	17,150 ± 23	64.69 ± 0.00
Predictive	207 ± 74	838 ± 432	1,237 ± 482	13.97 ± 0.06	17,179 ± 68	69.46 ± 0.00
predictive vs best +29.5 % vs Resource-aware, p 0.270 +38.0 % vs Resource-aware, p 0.347 +19.3 % vs Resource-aware, p 0.467 -0.2 % vs Random, p 0.371 +0.2 % vs Random, p 0.371 +98.9 % vs Random, p < 0.001						

Latency distribution

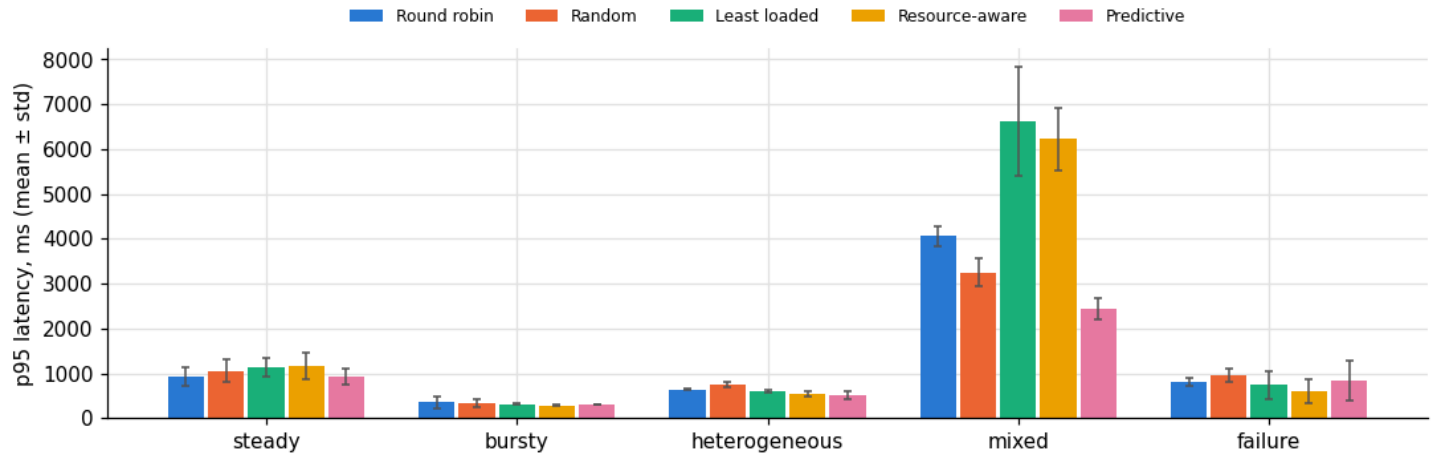
Every completed task of every repetition, per strategy (CDF, log scale).



Throughput

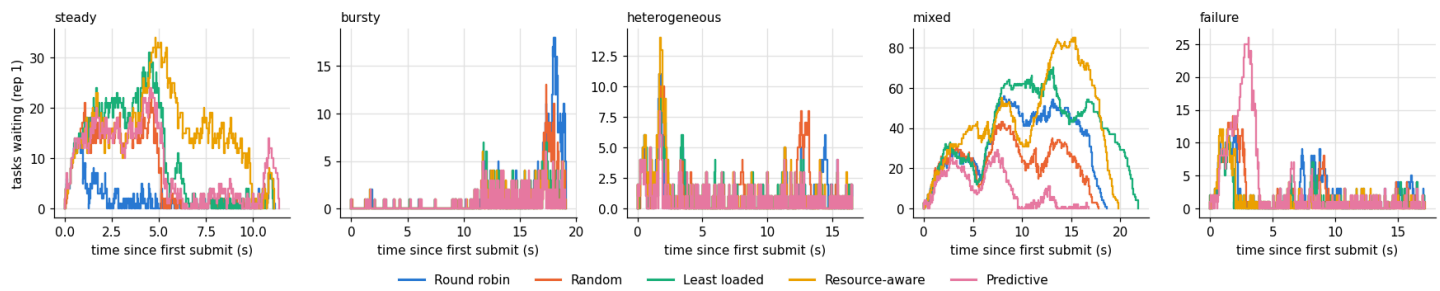


p95 latency

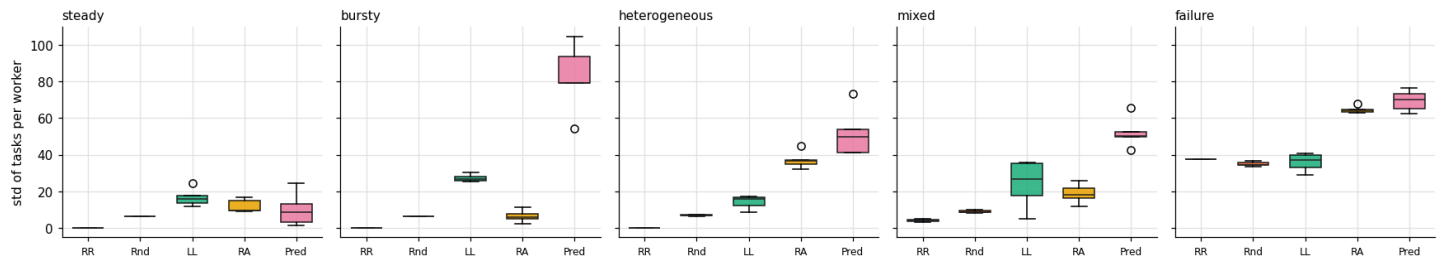


Queue over time

Tasks submitted but not yet started, repetition 1 of each strategy.

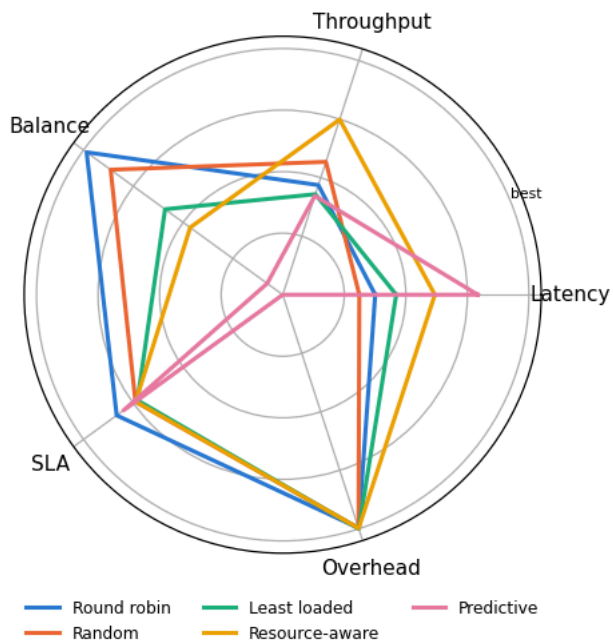


Worker imbalance



Overall profile

Each axis scales the strategies' means to [0, 1] within a scenario (1 = best), averaged over scenarios. A summary, not a test: read the tables for the numbers.



Scheduling overhead and prediction error

Strategy	Decision time mean (μ s)	Decision time p95 (μ s)
Round robin	91	139
Random	92	141
Least loaded	69	97
Resource-aware	83	138
Predictive	7,716	16,065

Scenario	Live exec-time MAE of predictive (ms)
----------	---------------------------------------

steady	41.7
bursty	21.7
heterogeneous	24.9
mixed	79.5
failure	35.7

Limitations

- Everything runs on one 4-core laptop: workers share the CPU, so "heterogeneous" hardware is simulated with pool sizes and a slowdown factor, and the prediction server competes with the workers it predicts.
- Five repetitions per cell: small differences can go undetected, and one test per metric and scenario is reported without a multiple-comparison correction.
- The models were trained on one campaign (prompt 15) on this same machine; the queue-forecast model never beat its baseline, and the overload model's threshold did not transfer to the held-out pattern (docs/components/ml-models.md).
- ML predictions can be wrong, and a poor model can make scheduling worse than a simple baseline; prediction calls add overhead to every decision (see above).
- Historical data may not capture sudden, unseen spikes; workload patterns drift, so models need retraining (prompt 21).
- New task types face a cold start (a type-average fallback); monitoring and heartbeats add their own overhead.
- A single active scheduler is a throughput ceiling; these results say nothing about larger clusters.